

Projekt – Saper: interface graficzny

Link do repozytorium: <https://gitlab-stud.elka.pw.edu.pl/ewojtiuk/pipr-projekt.git>

Projekt Saper autorzy: Emilia Wojtiuk, Julita Wasilewska

Saper: interface graficzny Autorstwa:

Julity Wasilewskiej

Opis:

Saper jest prostą grą komputerową stworzoną w 1981 roku, w którą prawie każdy właściciel komputera grał. Sama w sobie jest niezwykle prosta w obsłudze jak i w graniu, ponieważ głównym zadaniem jest odkrywanie zakrytych pól.

Założenia gry:

- Gracz ustawia wielkość planszy oraz ilość min
- Gracz odkrywa kolejne zakryte pola do zakończenia rozgrywki
- Na każdym z odkrytych pól napisana jest liczba min, które bezpośrednio stykają się z danym odkrytym polem (1~8).
- Przy trafieniu na liczbę zostanie odkryte tylko wybrane pole.
- W przypadku pola wokół którego nie znajduje się żadna miny (0), zostaną odkryte wszystkie sąsiednie puste pola (0), włącznie z najbliższym graniczącym polem z liczbą (1~8).
- Istnieje możliwość zabezpieczenia pola przed odkryciem przy użyciu flagi
- Przy przerwaniu rozgrywki jest możliwość zapisania aktualnego stanu gry i późniejszego jego wczytania

Zakończenie rozgrywki:

- Odkrycie miny skutkuje natychmiastową przegraną
- Aby wygrać należy odkryć wszystkie pola niebędące miną, bądź oflagować wszystkie miny.

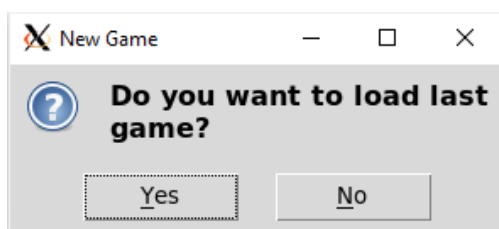
Interface graficzny projektu:

Wykorzystywane pliki:

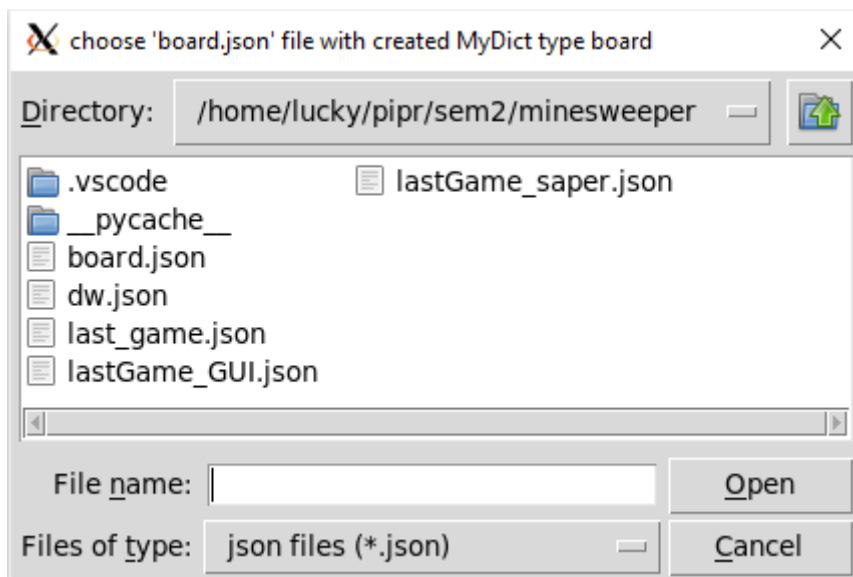
- saper_board.py
- grafika z folderu board_pics

Interface graficzny użytkownika został zaimplementowany przy użyciu biblioteki TKINTER.

Po uruchomieniu programu wyświetli się użytkownikowi okno dialogowe pytające o ewentualne wczytanie poprzedniej rozgrywki.



Program tworzy obiekt klasy MyDict, która zapisuje słownik na którym operuje program w pliku board.json wygenerowanym przez program, który należy otworzyć przy każdym rozpoczęciu nowej rozgrywki.

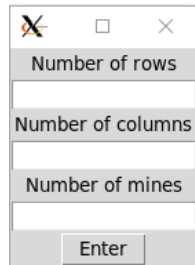


Tworząc nową planszę program tworzy tablicę przycisków w którym każdy zapisany jest w postaci słownika opisującego jego stan i własności:

1. współrzędne na planszy
2. obrazem graficznie reprezentujący pole
3. wartość, którą odczytuje z planszy utworzonej przez klasę Dict
4. stan informujący czy pole zostało już odkryte albo oflagowane (akcja użycia odpowiedniego przycisku myszy)

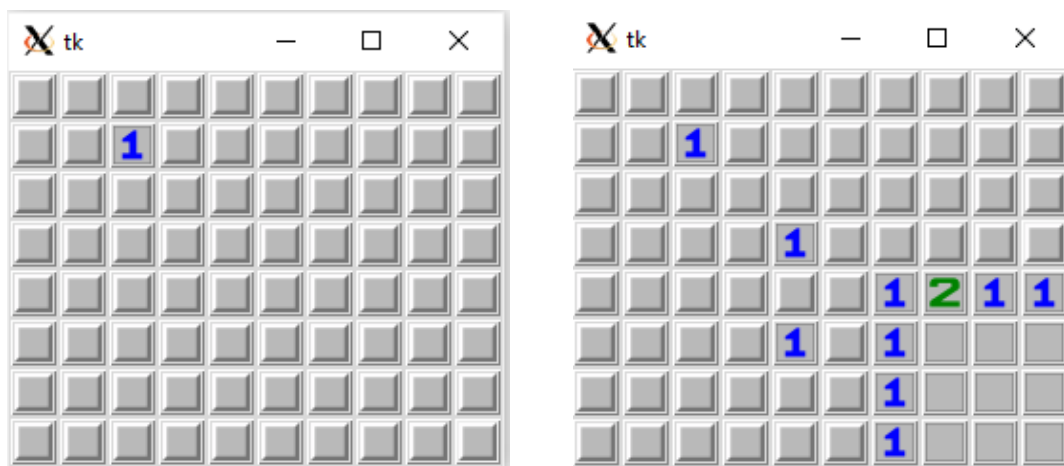
5. informację o ustawionej fladze

Po wybraniu opcji nowej gry użytkownik zostanie poproszony o wpisanie parametrów gry z przedziału [5, 30] dla wierszy i kolumn, w przypadku gdy zostanie wpisana błędna wartość lub ilość min będzie większa niż liczba pól gra się nie włączy a program będzie czekał na wpisanie poprawnej wartości.

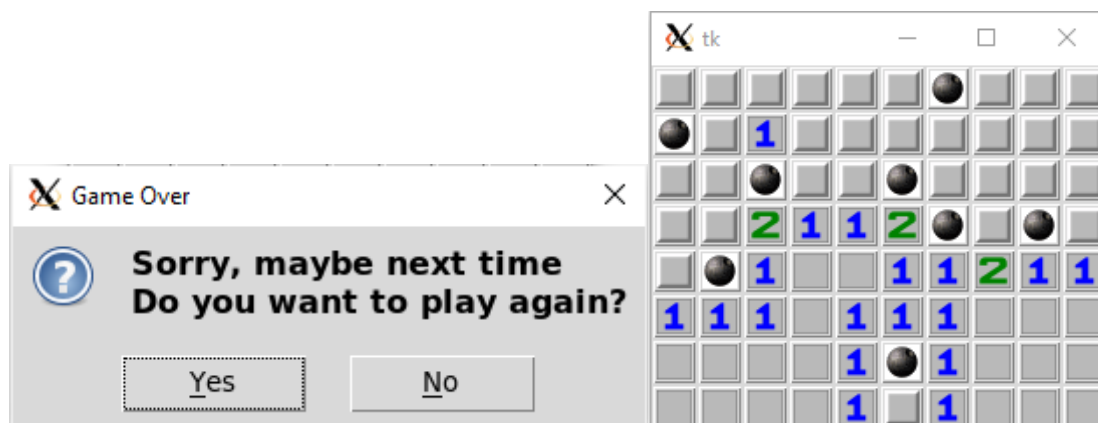


Forma do wprowadzenia parametrów gry. Zawiera trzy pola tekstowe: "Number of rows", "Number of columns" i "Number of mines". Poniżej znajduje się przycisk "Enter".

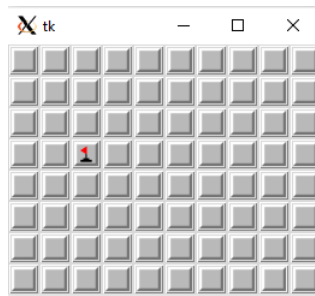
Użytkownik odkrywa kolejne pola naciskając na miejsce lewym przyciskiem myszy, co powoduje 'odkrycie' pola przez zmianę jego parametrów. W przypadku gdy pole jest puste (wartość 0) rekurencyjnie we wszystkich kierunkach odkrywane są kolejne puste pola do najbliższego pola z liczbą.



Gdy odkryte pole jest miną, odkryte zostają wszystkie zaminowane pola, a użytkownik zostaje poinformowany o porażce z możliwością nowej gry.

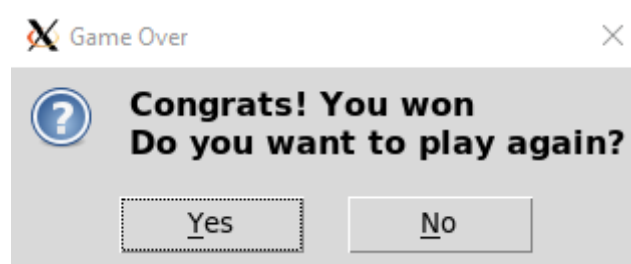


Prawy przycisk myszy ustawia flagę na wybrane pole pod warunkiem, że nie zostało jeszcze odkryte.



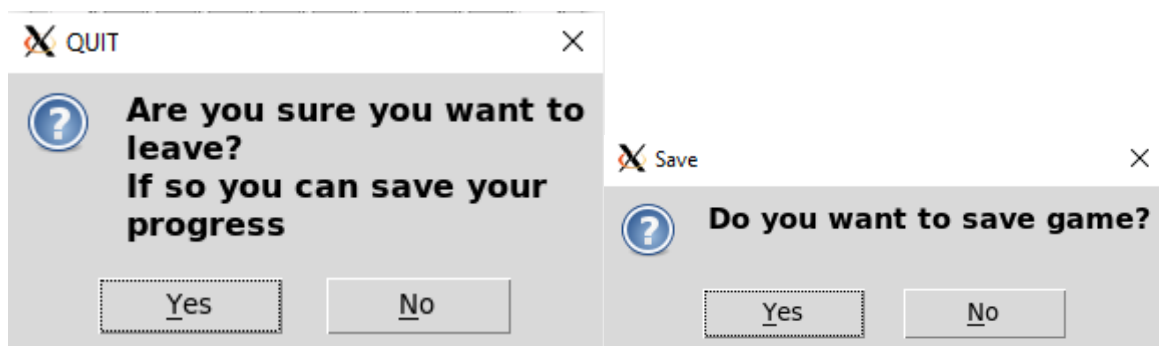
Oflagowane pole da się odkryć tylko po wcześniejszym usunięciu flagi (ponowne naciśnięcie prawym przyciskiem myszy).

Po każdym ruchu, jeśli nie została odkryta bomba, program sprawdza, czy nie był to ruch kończący rozgrywkę (czy wszystkie pola bez miny nie zostały już odkryte lub czy wszystkie miny nie zostały oflagowane – liczba postawionych flag nie może być większa od liczby min).

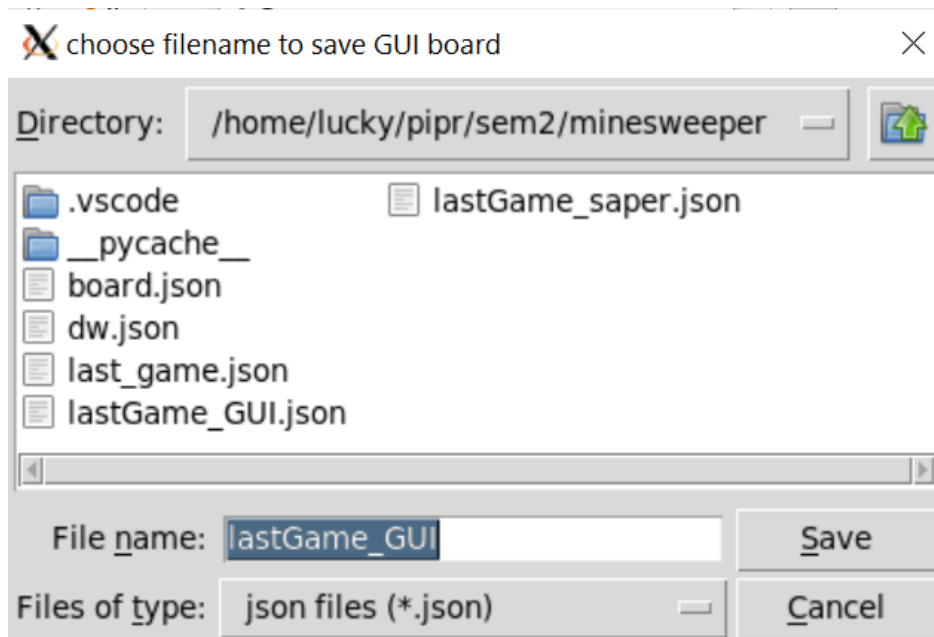


W przypadku wygranej i przegranej wyświetlane jest okno dialogowe z możliwością zamknięcia programu lub uruchomienia nowej gry.

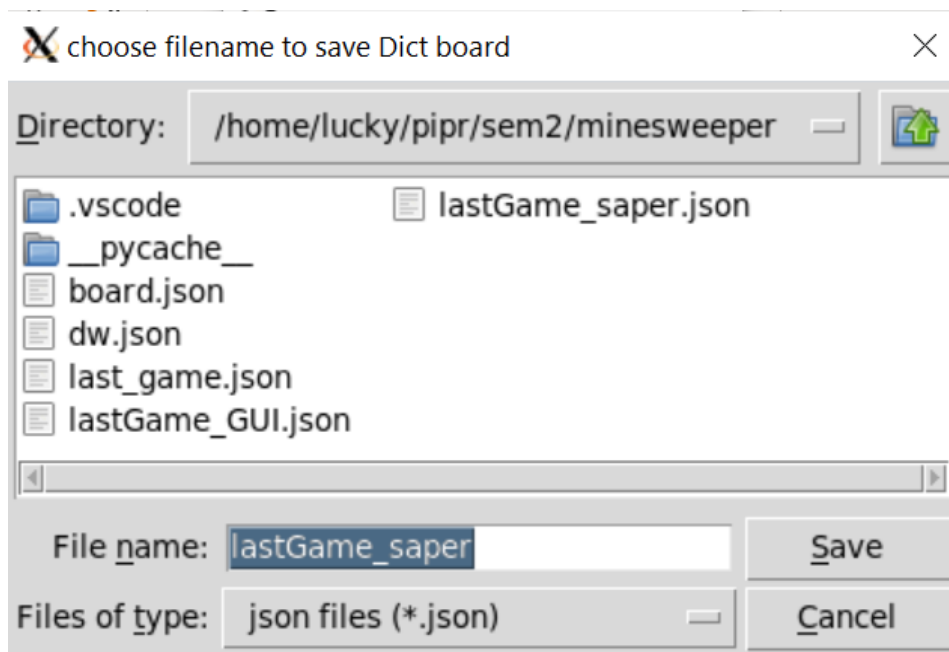
W przypadku zamknięcia okna z grą (naciśnięcie 'X') w trakcie rozgrywki wyświetlane jest okno dialogowe z możliwością powrotu do gry, a przy potwierdzeniu chęci wyjścia, kolejne okno z możliwością zapisania stanu gry do późniejszego wczytania.



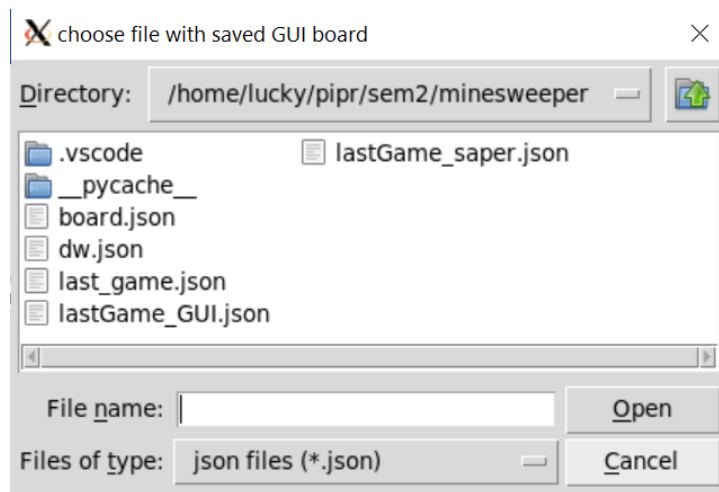
Zapisując rozgrywkę użytkownik najpierw zostanie poproszony o zapisanie słownika z klasy Game:



A następnie o zapisanie obiektu klasy MyDict wygenerowanego dla tej rozgrywki:



Przy wybraniu opcji wczytania starej rozgrywki użytkownik zostanie poproszony najpierw o otwarcie pliku ze słownikiem generowanym przez klasę Game, a następnie o plik z zapisanym słownikiem klasy MyDict.



Zaimplementowane metody klasy Game:

- *parameters* - przetworzenie przypisanych przez użytkownika danych
- *get_value* - sprawdza czy wpisane zostały poprawne wartości i przypisuje je do zmiennych
- *load_new_game* - okno dialogowe pytające o wczytanie gry lub zaczęcie nowej
- *load_game* - odczytuje pliki ze starą rozgrywką i tworzy z nich planszę
- *board* - tworzy planszę nowej gry
- *uncover* - odkrywa naciśnięte pole jeśli nie jest bombą
- *right_click_action* - określa co stanie się po naciśnięciu prawego przycisku myszy
- *left_click_action* - określa co stanie się po naciśnięciu lewego przycisku myszy
- *check_if_won_flags* - sprawdza czy użytkownik spełnił warunki wygranej przez flagi
- *check_if_won_uncover* - sprawdza czy użytkownik odkrył wszystkie pola bez bomb
- *new_board* - tworzy nową planszę typu MyDict
- *game_over* - kończy grę
- *file_save* - zapisuje pliki potrzebne do wczytania gry
- *close* - okno dialogowe przy zamykaniu okna programu

Wszystkie założenia zostały zrealizowane